

SHAKTI C-Class Performance Model Notes

perf-model

August 7, 2026

1 Goal

The model is a Python cycle-level performance model for the single-issue SHAKTI C-Class RISC-V core. It follows the CVA6 performance-model methodology: simulate performance, not architectural behavior. The model consumes the dynamic instruction stream that the RTL actually committed, then predicts the commit cycle for each instruction.

The current calibrated CoreMark result is:

Metric	Value
CoreMark iterations	40
RTL cycles from <code>app_log</code>	14,965,227
Model cycles	14,854,777
Cycle delta	-110,450 (-0.738%)
Inter-commit delta accuracy	99.233944%
Matched intervals	12,666,565 / 12,764,347

The held-out Dhrystone result after CoreMark calibration is 99.655836% inter-commit delta accuracy, with model cycles 167,113 versus 167,827 RTL cycles.

2 RTL Trace Instrumentation

The original C-Class commit log did not include cycle numbers. Cycle accuracy requires a timestamp on each committed instruction, so a minimal testbench-only change was made to `test_soc/c64_c32/TbSoc.bsv`. The core datapath was not modified.

Each commit line is emitted as:

```
cycle <cycle_count> core 0: ...
```

This produces the RTL commit cycle for every dynamic instruction. Benchmark-level cycle and instruction counts were checked after instrumentation to confirm that the tracing change was non-perturbing.

3 Trace Parser

`trace.py` parses the RTL `rtldump` commit log into `TraceEntry` objects. Each entry records:

- dynamic instruction index;
- program counter;
- instruction encoding;
- RTL commit cycle;

- privilege mode;
- integer/floating register writes;
- CSR writes;
- memory addresses when present;
- inferred actual next PC.

The actual next PC is inferred from the next committed instruction. That is enough to classify branches as taken or fallthrough without computing data values.

4 Timing Decoder

`isa.py` decodes RV64IMAFDC only far enough for timing. The model does not need to know that an add computes a sum, but it does need to know which registers are read and written, which functional unit is used, whether an instruction is a load, store, branch, jump, multiply, divide, CSR instruction, or trap, and whether the instruction is 16-bit compressed or 32-bit.

For example, the timing metadata for a load must say:

```
reads x<base>
writes x<rd>
is_load = true
functional unit = memory
result is not immediately bypassable from execute
```

This metadata drives scoreboard locking, RAW hazard checks, bypass decisions, functional-unit latency, and front-end control timing.

Compressed decode was a concrete accuracy issue. RV64C integer doubleword instructions such as `c.ld`, `c.sd`, `c.ldsp`, and `c.sdsp` must not be confused with floating compressed memory operations such as `c.fld`, `c.fsd`, `c.fldsp`, and `c.fsdsp`. If an integer load is decoded as a floating load, the model locks the wrong register file and misses real integer RAW dependencies.

Example:

```
c.ld    x10, 0(x8)
addi    x11, x10, 1
```

Correct decode sees a dependency from `x10` to the following `addi`. Incorrect decode as `c.fld` would mark the producer as writing `f10`, so the model would issue the `addi` too early.

5 Pipeline Model

The SHAKTI C-Class is not modeled as CVA6. The useful CVA6 lesson is the simulation methodology, not the microarchitecture. C-Class is strictly in-order and has six major stages:

Stage	Role
0	PC generation and branch prediction
1	instruction fetch response and compressed decompression
2	decode and register read
3	execute
4	memory and late execute routing
5	writeback, commit, CSR, and trap handling

The RTL uses inter-stage buffers (ISBs). Each stage fires when its input is not empty, its output is not full, and its local rule guards are true. This means backpressure is structural: a full downstream queue stalls the producer.

The model represents ISBs as bounded queues:

`q_s0s1, q_s1s2, q_s2s3, q_s3s4, q_s4s5`

Their depths are derived from `makefile.inc`, not guessed. This matters because changing an ISB depth changes when a local stall propagates backward through the front end.

One model cycle executes stages in reverse order:

```
commit
stage4
execute
decode
fetch/decompress
fetch
```

This approximates synchronous hardware behavior. Downstream stages consume first, then upstream stages can observe freed queue space in the same modeled cycle. A forward-order software loop would create artificial stalls.

6 Modeled Hazards

The model explicitly includes:

- structural hazards from full ISBs;
- in-order issue and commit;
- scoreboard register locks;
- scoreboard release at commit;
- bypass from the heads of the two later ISBs;
- fixed load/store cache-hit latency;
- multiplier and divider latency from `MULSTAGES_TOTAL` and `DIVSTAGES`;
- branch misprediction refill penalty;
- CSR/trap/fence-style writeback flush penalty.

The scoreboard is modeled as a per-register lock array, matching C-Class rather than CVA6's out-of-order scoreboard FIFO. Destination registers are locked at execute and released at commit. With `no_wawstalls`, the model tracks a small WAW identifier so a late release does not unlock a newer writer.

7 Branch Predictor

Most residual timing errors were control-flow related, so the branch predictor had to be modeled in detail. The predictor model includes:

- BTB entries;
- BHT saturating counters;
- gshare global history;
- return-address stack when RTL is built with `bpu_ras`;
- compressed lower/upper halfword matching through the BTB `hi` bit;
- delayed predictor training;
- global-history restore after misprediction.

The largest CoreMark accuracy jumps came from:

- fixing compressed integer memory decode;
- matching BTB `hi` during prediction;
- reproducing BSV hash width behavior for gshare;
- applying predictor training one model cycle after execute;
- deriving RAS enablement from `makefile.inc`.

The gshare hash fix was subtle. In BSV, shifting a `Bit#(histbits)` value keeps that narrow width before widening. Python integers are unbounded, so the model had to explicitly mask the shifted history value to reproduce RTL indexing.

8 Accuracy Metric

The model uses the CVA6 inter-commit spacing metric. For dynamic instruction i :

$$\Delta t_{\text{RTL}}(i) = t_{\text{RTL}}(i) - t_{\text{RTL}}(i - 1)$$

$$\Delta t_{\text{model}}(i) = t_{\text{model}}(i) - t_{\text{model}}(i - 1)$$

$$\text{accuracy} = \frac{|\{i : \Delta t_{\text{model}}(i) = \Delta t_{\text{RTL}}(i)\}|}{|\{i\}|}$$

This metric does not let one early offset hide later local timing mistakes.

9 What Did Not Help

Several plausible fixes were tested and rejected:

- disabling RAS reduced CoreMark accuracy to roughly 99.209439%;
- adding a global extra cycle to mispredicts reduced accuracy to 98.664530%;
- a coarse wrong-path front-end occupancy model also reduced accuracy to 98.664530%.

These negative results show why the current model should not be tuned with broad constants. The remaining mismatches are conditional front-end/control effects.

10 Dual-Issue Prediction Mode

The dual-issue mode now models the actual SHAKTI dual-issue branch rather than an ideal independent-lane machine. The important parameters extracted from the dual branch are:

- `num_issue=2`;
- 64-bit fetch path;
- `instr_queue=6` for the stage1-to-stage2 MIMO queue, not the apparent `isb_s1s2=2`;
- one vector bundle slot for stage2-to-stage3, represented internally as two scalar trace entries;
- eight vector bundle slots for stage3-to-stage4 and stage4-to-stage5, represented internally as sixteen scalar trace entries;
- two ALUs;
- one shared branch/control unit;
- one memory issue path;
- one mul/div path;
- one FPU path;
- no intra-bundle forwarding;
- `dual_mem` disabled and effectively unwired.

Pairing is not arbitrary. The model implements the stage2 whitelist:

- ALU may pair with ALU, MULDIV, FLOAT, MEMORY, or CONTROL;
- CONTROL may pair with MULDIV, FLOAT, or MEMORY;
- MEM+MEM is disabled because `dual_mem` is not compiled;
- MULDIV+MULDIV, FLOAT+FLOAT, MEM+MULDIV, MEM+FLOAT, MULDIV+FLOAT, BRANCH+BRANCH, and anything with SYSTEM/TRAP/WFI are single-issued.

The model also implements the non-obvious pair hazards. Intra-bundle RAW from the first fetched instruction to the second fetched instruction blocks pairing. WAW and WAR pairs are allowed because `no_wawstalls` is defined. Once a pair is formed, it advances in lockstep: if either slot's operands, functional unit, or stage4 result are not ready, neither slot advances. Stage5 commits the pair atomically, so both instructions receive the same predicted commit cycle.

These dual-issue outputs are predictions, not validated accuracy results, because the current input trace is still the single-issue committed instruction stream. A dual RTL trace is needed before an inter-commit accuracy number can be reported for the dual branch.

11 Known Limitations

The model deliberately does not capture:

- data-cache misses or cache conflicts;
- instruction-cache misses;

- data-dependent divider timing;
- CSR/trap behavior requiring data values;
- fully accurate wrong-path front-end queue occupancy after control redirects.